LAPORAN PRATIKUM

DISUSUN OLEH :

BILQIS RIZ FEEBRIANI BR GAJAH
2025573010044

Kelas : TI 1C
Mata Kuliah : Logika dan Struktur Data
Dosen : Muhammad Reza Zulman, S.ST.,M.Sc

PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER
POLITEKNIK NEGERI LHOKSEUMAWE
2025-2026

KATA PENGANTAR

Puji syukur kehadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum ini dengan baik. Laporan ini disusun sebagai salah satu bentuk tugas dalam mata kuliah Logika dan Struktur Data di Program Studi Teknik Informatika, Politeknik Negeri Lhokseumawe.

Dalam laporan ini, penulis membahas mengenai konsep kompleksitas waktu (time complexity) dan kompleksitas ruang (space complexity), serta implementasi algoritma menggunakan bahasa pemrograman JavaScript. Selain itu, laporan ini juga memuat latihan dan tugas yang bertujuan untuk memahami serta menganalisis efisiensi suatu algoritma.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang.

Akhir kata, semoga laporan ini dapat bermanfaat bagi penulis maupun pembaca.

DAFTAR ISI

KATA PENGANTAR i
DAFTAR ISI ii
BAB I 1
PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan Praktikum 1
BAB II 2
PROSEDUR KERJA 2
2.1 Langkah-Langkah Praktikum 2
2.2 Latihan 7
2.3 Tugas 9
BAB III 13
PENUTUP 13
3.1 Kesimpulan 13

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, efisiensi algoritma merupakan hal yang sangat penting, terutama ketika berhadapan dengan data dalam jumlah besar. Kompleksitas waktu (Big O Notation) digunakan untuk mengukur seberapa cepat suatu algoritma berjalan, sedangkan kompleksitas ruang digunakan untuk mengetahui seberapa besar memori yang digunakan.Melalui praktikum ini, mahasiswa diharapkan dapat memahami berbagai jenis kompleksitas algoritma, seperti O(1), O(n), O(n^2), dan O(log n), serta mampu membandingkan performa antar algoritma. Selain itu, mahasiswa juga dilatih untuk melakukan refactor kode agar menjadi lebih efisien dan optimal

1.2 Tujuan Praktikum

1. Memahami konsep dasar kompleksitas waktu (time complexity) dan ruang (space complexity).
2. Mampu mengidentifikasi kompleksitas dari berbagai algoritma.
3. Mampu mengimplementasikan algoritma dengan efisiensi yang lebih baik.
4. Membandingkan performa beberapa algoritma berdasarkan waktu eksekusi.
5. Melatih kemampuan analisis dan refactoring kode program.

BAB II

PROSEDUR KERJA

2.1 Langkah-Langkah Praktikum

1. Buat folder praktikum-5

```
mkdir praktikum-5
```

```
cd praktikum-5
```

```
User@DESKTOP-VQROPFC MINGW64 /d/2025573010044_algoritma_dan_struktur_data (main)
$ mkdir praktikum-5

User@DESKTOP-VQROPFC MINGW64 /d/2025573010044_algoritma_dan_struktur_data (main)
$ cd praktikum-5
```

2. Inisialisasi npm project.

```
npm init -y
```

```
User@DESKTOP-VQROPFC MINGW64 /d/2025573010044_algoritma_dan_struktur_data/praktikum-5 (main)
$ npm init -y
Wrote to D:\2025573010044_algoritma_dan_struktur_data\praktikum-5\package.json:

{
  "name": "praktikum-5",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

3. Buat file baru bernama 01-intro-kompleksitas.js.

```
touch 01-intro-kompleksitas.js
```

JS 01-intro-kompleksitas.js

4. Buka file 01-intro-kompleksitas.js di VS Code, lalu ketikkan kode berikut.

```
// 01-intro-kompleksitas.js
// Mengukur perbedaan performa algoritma secara empiris

// Fungsi bantu: ukur waktu eksekusi
function ukurWaktu(label, fn) {
    const mulai = Date.now();
    fn();
    const selesai = Date.now();
    console.log(`${label}: ${selesai - mulai} ms`);
}
```

```javascript
const N = 100_000; // ukuran data: 100 ribu

// --- Algoritma A: O(n) — loop satu kali ---
function jumlahkanLinear(n) {
    let total = 0;
    for (let i = 1; i <= n; i++) total += i;
    return total;
}

// --- Algoritma B: O(1) — rumus matematika ---
function jumlahkanRumus(n) {
    return (n * (n + 1)) / 2;
}

// --- Algoritma C: O(n²) — loop bersarang ---
function cariPasangan(arr) {
    const pasangan = [];
    for (let i = 0; i < arr.length; i++) {
        for (let j = i + 1; j < arr.length; j++) {
            if (arr[i] + arr[j] === 10) pasangan.push([arr[i], arr[j]]);
        }
    }
    return pasangan;
}

const data = Array.from({length: 5000}, (_, i) => i);

console.log('=== Perbandingan Waktu Eksekusi ===');
ukurWaktu('O(1)  jumlahkanRumus  ', () => jumlahkanRumus(N));
ukurWaktu('O(n)  jumlahkanLinear ', () => jumlahkanLinear(N));
ukurWaktu('O(n²) cariPasangan    ', () => cariPasangan(data));

console.log('\nHasil sama?', jumlahkanLinear(100) === jumlahkanRumus(100));
```

```javascript
function ukurWaktu(label, fn){
    const mulai = Date.now();
    fn();
    const selesai = Date.now();
    console.log(`${label}: ${selesai - mulai} ms`);

}
const N = 100_000;

function jumlahkanLinear(n) {
    let total = 0;
    for (let i = 1; i <= n; i++);
    return total;

}
function jumlahkanRumus(n){
    return (n * (n + 1)) / 2;

}
function cariPasangan(arr){
    const pasangan =[];
    for (let i = 0; i > arr.length; i++){
        for(let j = i + 1; j <arr.length; j++) {
            if (arr[i] + arr [j] === 10 ) pasangan.push([arr[i], arr[j]]);
            }
    }
    return pasangan;

}
const data = Array.from({ length: 5000}, (_,i) => i);

console.log("=== Perbandingan Waktu Eksekusi ===");
ukurWaktu("0(1) jumlahkanRumus ",() => jumlahkanRumus(N));
ukurWaktu("0(n) jumlahkanLinear ",() => jumlahkanLinear(N));
ukurWaktu("0(n^2) cariPasangan ",() => cariPasangan(data));

console.log('\nHasil sama?', jumlahkanLinear(100) === jumlahkanRumus(100));
```

5. Simpan file, lalu jalankan dari terminal.

```
User@DESKTOP-VQROPFC MINGW64 /d/2025573010044_algoritma_dan_struktur_data (main)
$ cd praktikum-5

User@DESKTOP-VQROPFC MINGW64 /d/2025573010044_algoritma_dan_struktur_data/praktikum-5 (main)
$ node 01-intro-kompleksitas.js
=== Perbandingan Waktu Eksekusi ===
0(1) jumlahkanRumus : 0 ms
0(n) jumlahkanLinear : 2 ms
0(n^2) cariPasangan : 0 ms

Hasil sama? false
```

6. Buat file baru bernama 02-kelas-big-o.js

`touch 02-destructuring-spread.js`

JS 02-kelas-big-o.js

7. Buka file 02-kelas-big-o.js di VS Code, lalu ketikkan kode berikut.

```javascript
// 02-kelas-big-o.js
// Contoh konkret setiap kelas kompleksitas Big O

// ── O(1) — Konstan ──────────────────────────────
function ambilPertama(arr)  { return arr[0]; }          // O(1)
function tambahkanItem(arr, item) { arr.push(item); }   // O(1)
function isGenap(n) { return n % 2 === 0; }             // O(1)

// ── O(log n) — Logaritmik ───────────────────────
function binarySearch(arr, target) {
    let kiri = 0, kanan = arr.length - 1;
    let langkah = 0;
    while (kiri <= kanan) {
        langkah++;
        const tengah = Math.floor((kiri + kanan) / 2);
        if (arr[tengah] === target) {
```

```
            console.log('  Ditemukan di indeks ${tengah} setelah ${langkah} langkah');
            return tengah;
        }
        if (arr[tengah] < target) kiri = tengah + 1;
        else kanan = tengah - 1;
    }
    return -1;
}

// ── O(n) - Linear ──────────────────────────────
function cariMax(arr) {
    let maks = arr[0];
    for (const x of arr) if (x > maks) maks = x;
    return maks;
}

// ── O(n²) - Kuadratik ──────────────────────────
function bubbleSortNaif(arr) {
    const a = [...arr];
    for (let i = 0; i < a.length; i++)
        for (let j = 0; j < a.length - i - 1; j++)
            if (a[j] > a[j+1]) [a[j], a[j+1]] = [a[j+1], a[j]];
    return a;
}

// ── O(2ⁿ) - Eksponensial ──────────────────────
function fibRekursif(n) {
    if (n <= 1) return n;
    return fibRekursif(n-1) + fibRekursif(n-2); // dua panggilan rekursif!
}

// ── Demonstrasi ────────────────────────────────
console.log('=== O(1) - selalu cepat ===');
console.log(ambilPertama([10,20,30,40,50])); // 10
console.log(isGenap(42));                    // true

console.log('\n=== O(log n) - Binary Search ===');
const sorted = Array.from({length:1_000_000},(_,i)=>i);
binarySearch(sorted, 731_452); // cari di 1 juta data

console.log('\n=== O(n) - Linear Search ===');
console.log('Max dari 1000 elemen:',
cariMax(Array.from({length:1000},()=>Math.random()*1000|0)));

console.log('\n=== O(n²) - Bubble Sort (kecil saja) ===');
console.log(bubbleSortNaif([64, 34, 25, 12, 22, 11, 90]));

console.log('\n=== O(2ⁿ) - Fibonacci Rekursif (n kecil!) ===');
for (let i = 0; i <= 10; i++) process.stdout.write(fibRekursif(i) + ' ');
console.log('');

// Demonstrasi betapa lambatnya O(2ⁿ)
console.log('\nWaktu fib(35) O(2ⁿ):');
let t = Date.now(); fibRekursif(35); console.log(Date.now()-t, 'ms');

console.log('Waktu fib(35) O(n) memoization:');
const memo = {};
function fibMemo(n) {
    if (n <= 1) return n;
    if (memo[n]) return memo[n];
    return memo[n] = fibMemo(n-1) + fibMemo(n-2);
```

```
function ambilPertama(arr) {
  return arr[0];
}
function tambahkanItem(arr, item) {
  arr.push(item);
}
function isGenap(n) {
  return n % 2 === 0;
}
function binarySearch(arr, target) {
  let kiri = 0,
    kanan = arr.length - 1;
  let langkah = 0;
  while (kiri <= kanan) {
    langkah++;
    const tengah = Math.floor((kiri + kanan) / 2);
    if (arr[tengah] === target) {
      console.log(`  Ditemukan di indeks ${tengah} setelah ${langkah} langkah`);
      return tengah;
    }
    if (arr[tengah] < target) kiri = tengah + 1;
    else kanan = tengah - 1;
  }
  return -1;
}
function cariMax(arr) {
  let maks = arr[0];
  for (const x of arr) if (x > maks) maks = x;
  return maks;
}
function bubbleSortNaif(arr) {
  const a = [...arr];
  for (let i = 0; i < a.length; i++)
    for (let j = 0; j < a.length - i - 1; j++)
      if (a[j] > a[j + 1]) [a[j], a[j + 1]] = [a[j + 1], a[j]];
  return a;
}
function fibRekursif(n) {
  if (n <= 1) return n;
  return fibRekursif(n - 1) + fibRekursif(n - 2);
}
console.log("=== O(1) — selalu cepat ===");
console.log(ambilPertama([10, 20, 30, 40, 50]));
console.log(isGenap(42));
console.log("\n=== O(log n) — Binary Search ===");
const sorted = Array.from({ length: 1_000_000 }, (_, i) => i);
binarySearch(sorted, 731_452);
console.log("\n=== O(n) — Linear Search ===");
console.log(
  "Max dari 1000 elemen:",
  cariMax(Array.from({ length: 1000 }, () => (Math.random() * 1000) | 0)),
);
console.log("\n=== O(n2) — Bubble Sort (kecil saja) ===");
console.log(bubbleSortNaif([64, 34, 25, 12, 22, 11, 90]));
console.log("\n=== O(2n) — Fibonacci Rekursif (n kecil!) ===");
for (let i = 0; i <= 10; i++) process.stdout.write(fibRekursif(i) + " ");
console.log("");

console.log("\nWaktu fib(35) O(2n):");
let t = Date.now();
fibRekursif(35);
console.log(Date.now() - t, "ms");
console.log("Waktu fib(35) O(n) memoization:");
const memo = {};
function fibMemo(n) {
  if (n <= 1) return n;
  if (memo[n]) return memo[n];
  return (memo[n] = fibMemo(n - 1) + fibMemo(n - 2));
}
t = Date.now();
fibMemo(35);
console.log(Date.now() - t, "ms");
```

```
}
t = Date.now(); fibMemo(35); console.log(Date.now()-t, 'ms');
```

8. Simpan file, lalu jalankan dari terminal.

```
User@DESKTOP-VQROPFC MINGW64 /d/2025573010044_algoritma_dan_struktur_data (main)
$ cd praktikum-5

User@DESKTOP-VQROPFC MINGW64 /d/2025573010044_algoritma_dan_struktur_data/praktikum-5 (main)
$ node 02-kelas-big-o.js
=== O(1) — selalu cepat ===
10
true

=== O(log n) — Binary Search ===
 Ditemukan di indeks 731452 setelah 19 langkah

=== O(n) — Linear Search ===
Max dari 1000 elemen: 998

=== O(n2) — Bubble Sort (kecil saja) ===
[
  11, 12, 22, 25,
  34, 64, 90
]

=== O(2n) — Fibonacci Rekursif (n kecil!) ===
0 1 1 2 3 5 8 13 21 34 55

Waktu fib(35) O(2n):
110 ms
Waktu fib(35) O(n) memoization:
1 ms

User@DESKTOP-VQROPFC MINGW64 /d/2025573010044_algoritma_dan_struktur_data/praktikum-5 (main)
$
```

9. Buat file baru bernama 03-space-complexity.js

```
touch 03-space-complexity.js
```

JS 03-space-complexity.js

10. Buka file 03-space-complexity.js di VS Code, lalu ketikkan kode berikut.

```js
// 04-space-complexity.js

// --- O(1) Space: hanya pakai variabel tambahan konstan ---
function jumlahArray(arr) {
    let total = 0;              // hanya 1 variabel tambahan
    for (const x of arr) total += x;
    return total;
}

// --- O(n) Space: membuat struktur baru sebesar input ---
function duplikasiArray(arr) {
    const baru = [];            // array baru tumbuh seiring arr
    for (const x of arr) baru.push(x * 2);
    return baru;
}

// --- O(n) Space: rekursi (call stack) ---
function faktorialRekursif(n) {
    if (n <= 1) return 1;
    return n * faktorialRekursif(n - 1); // n frame di call stack
}

// --- O(1) Space: faktorial iteratif ---
function faktorialIteratif(n) {
    let hasil = 1;
    for (let i = 2; i <= n; i++) hasil *= i; // hanya 2 variabel
    return hasil;
}

const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

console.log('Jumlah array   :', jumlahArray(arr));          // O(1) space
console.log('Duplikasi array:', duplikasiArray(arr));       // O(n) space
console.log('Faktorial 10 rekursif :', faktorialRekursif(10));
console.log('Faktorial 10 iteratif :', faktorialIteratif(10));

// Visualisasi: hitung elemen unik (O(n) space)
function hitungUnik(arr) {
    const seen = new Set(); // Set tumbuh hingga n elemen
```

```js
function jumlahArray(arr) {
  let total = 0;
  for (const x of arr) total += x;
  return total;
}

function duplikasiArray(arr) {
  const baru = [];
  for (const x of arr) baru.push(x * 2);
  return baru;
}

function faktorialRekursif(n) {
  if (n <= 1) return 1;
  return n * faktorialRekursif(n - 1);
}

function faktorialIteratif(n) {
  let hasil = 1;
  for (let i = 2; i <= n; i++) hasil *= i;
  return hasil;
}
const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
console.log("Jumlah array :", jumlahArray(arr));
console.log("Duplikasi array:", duplikasiArray(arr));
console.log("Faktorial 10 rekursif :", faktorialRekursif(10));
console.log("Faktorial 10 iteratif :", faktorialIteratif(10));

function hitungUnik(arr) {
  const seen = new Set();
  for (const x of arr) seen.add(x);
  return seen.size;
}
const dataAcak = [1, 2, 3, 2, 1, 4, 5, 3, 6, 4, 7];
console.log("Elemen unik:", hitungUnik(dataAcak));
```

```js
    for (const x of arr) seen.add(x);
    return seen.size;
}

const dataAcak = [1,2,3,2,1,4,5,3,6,4,7];
console.log('Elemen unik:', hitungUnik(dataAcak)); // 7
```

11. Simpan file, lalu jalankan dari terminal.

```
User@DESKTOP-VQROPFC MINGW64 /d/2025573010044_algoritma_dan_struktur_data (main)
$ cd praktikum-5

User@DESKTOP-VQROPFC MINGW64 /d/2025573010044_algoritma_dan_struktur_data/praktikum-5 (main)
$ node 03-space-complexity.js
Jumlah array : 55
Duplikasi array: [
   2,  4,  6,  8, 10,
  12, 14, 16, 18, 20
]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : 3628800
Elemen unik: 7

User@DESKTOP-VQROPFC MINGW64 /d/2025573010044_algoritma_dan_struktur_data/praktikum-5 (main)
$
```

12. Buat file baru bernama 04-refactor.js

```
touch 04-refactor.js
```

JS 04-refactor.js

13.Buka file 04-refactor.js di VS Code, lalu ketikkan kode berikut.

```javascript
// 05-refactor.js — Refactoring kode berperforma buruk

// ═══════════════════════════════════════════════════
// SKENARIO 1: Cek duplikat dalam array
// ═══════════════════════════════════════════════════

// BURUK: O(n²) — nested loop
function adaDuplikatLambat(arr) {
    for (let i = 0; i < arr.length; i++)
        for (let j = i+1; j < arr.length; j++)
            if (arr[i] === arr[j]) return true;
    return false;
}

// BAIK: O(n) — gunakan Set
function adaDuplikatCepat(arr) {
    const seen = new Set();
    for (const x of arr) {
        if (seen.has(x)) return true;
        seen.add(x);
    }
    return false;
}

// ═══════════════════════════════════════════════════
// SKENARIO 2: Frekuensi kemunculan elemen
// ═══════════════════════════════════════════════════

// BURUK: O(n²) — indexOf di dalam loop
function frekuensiLambat(arr) {
    const unik = [];
    const hitung = [];
    for (const x of arr) {
        const idx = unik.indexOf(x);   // O(n) di dalam loop O(n)
        if (idx === -1) { unik.push(x); hitung.push(1); }
        else hitung[idx]++;
    }
    return Object.fromEntries(unik.map((u,i) => [u, hitung[i]]));
}
```

```javascript
function adaDuplikatLambat(arr) {
  for (let i = 0; i < arr.length; i++)
    for (let j = i + 1; j < arr.length; j++) if (arr[i] === arr[j]) return true;
  return false;
}

function adaDuplikatCepat(arr) {
  const seen = new Set();
  for (const x of arr) {
    if (seen.has(x)) return true;
    seen.add(x);
  }
  return false;
}

function frekuensiLambat(arr) {
  const unik = [];
  const hitung = [];
  for (const x of arr) {
    const idx = unik.indexOf(x);
    if (idx === -1) {
      unik.push(x);
      hitung.push(1);
    } else hitung[idx]++;
  }
  return Object.fromEntries(unik.map((u, i) => [u, hitung[i]]));
}

function frekuensiCepat(arr) {
  const counter = {};
  for (const x of arr) counter[x] = (counter[x] || 0) + 1;
  return counter;
}

const besar = Array.from({ length: 20000 }, () =>
  Math.floor(Math.random() * 1000),
);

let t = Date.now();
adaDuplikatLambat(besar);
console.log("Duplikat LAMBAT O(n2):", Date.now() - t, "ms");
t = Date.now();
adaDuplikatCepat(besar);
console.log("Duplikat CEPAT O(n) :", Date.now() - t, "ms");
const kata = ["a", "b", "a", "c", "b", "a", "d"];
console.log("\nFrekuensi:", frekuensiCepat(kata));
```

```javascript
// BAIK: O(n) — gunakan object/Map sebagai counter
function frekuensiCepat(arr) {
    const counter = {};
    for (const x of arr) counter[x] = (counter[x] || 0) + 1;
    return counter;
}

// ── Benchmark ──────────────────────────────────
const besar = Array.from({length: 20000}, () => Math.floor(Math.random() * 1000));

let t = Date.now();
adaDuplikatLambat(besar);
console.log('Duplikat LAMBAT O(n²):', Date.now()-t, 'ms');

t = Date.now();
adaDuplikatCepat(besar);
console.log('Duplikat CEPAT  O(n) :', Date.now()-t, 'ms');

const kata = ['a','b','a','c','b','a','d'];
console.log('\nFrekuensi:', frekuensiCepat(kata));
```

14. Simpan file, lalu jalankan dari terminal.

```
User@DESKTOP-VQROPFC MINGW64 /d/2025573010044_algoritma_dan_struktur_data (main)
$ cd praktikum-5

User@DESKTOP-VQROPFC MINGW64 /d/2025573010044_algoritma_dan_struktur_data/praktikum-5 (main)
$ node 04-refactor.js
Duplikat LAMBAT O(n2): 0 ms
Duplikat CEPAT O(n) : 0 ms

Frekuensi: { a: 3, b: 2, c: 1, d: 1 }
```

2.2 Latihan

Latihan 1: Identifikasi Kompleksitas

1. Buat file baru latihan-1.js.

2. Untuk setiap potongan kode di bawah ini, tulis dalam komentar Big O-nya dan alasannya:

3. Fungsi A: function fn(n) { return n * 2; } → Big O: ?

4. Fungsi B: for (let i=0; i<n; i++) for (let j=0; j<n; j++) console.log(i,j); → Big O: ?

5. Fungsi C: for (let i=1; i<n; i*=2) console.log(i); → Big O: ?

6. Fungsi D: arr.forEach(x => arr.forEach(y => arr.forEach(z => console.log(x,y,z)))); → Big O: ?

7. Implementasikan dan jalankan setiap fungsi, ukur waktunya dengan Date.now() untuk n=1000.

8. Buat function hitungKompleksitas(n, fn) yang menjalankan fn(n) dan mencetak waktu eksekusi.

Codenya:

```
praktikum-5 > JS latihan-1.js > ...
function fn(n) {
  return n * 2;
}
function fungsiB(n) {
  let count = 0;

  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      count++;
    }
  }

  console.log("Total operasi:", count);
}
function fungsiC(n) {
  let i = 1;
  let count = 0;

  while (i < n) {
    count++;
    i *= 2;
  }

  console.log("Total iterasi:", count);
}
function fungsiD(arr) {
  let count = 0;

  arr.forEach((x) => {
    arr.forEach((y) => {
      arr.forEach((z) => {
        count++;
      });
    });
  });

  console.log("Total operasi:", count);
}
function hitungKompleksitas(n, fn) {
  const start = Date.now();
  fn(n);
  const end = Date.now();

  console.log("Waktu:", end - start, "ms");
}
const n = 50;

console.log("=== Fungsi A (O(1)) ===");
hitungKompleksitas(n, fn);

console.log("\n=== Fungsi B (O(n^2)) ===");
hitungKompleksitas(n, fungsiB);

console.log("\n=== Fungsi C (O(log n)) ===");
hitungKompleksitas(n, fungsiC);

console.log("\n=== Fungsi D (O(n^3)) ===");
const arr = [1, 2, 3, 4, 5];
const start = Date.now();
fungsiD(arr);
const end = Date.now();
console.log("Waktu:", end - start, "ms");
```

Hasil run:

```
User@DESKTOP-VQROPFC MINGW64 /d/2025573010044_algoritma_dan_struktur_data (main)
$ cd praktikum-5

User@DESKTOP-VQROPFC MINGW64 /d/2025573010044_algoritma_dan_struktur_data/praktikum-5 (main)
$ node latihan-1.js
=== Fungsi A (O(1)) ===
Waktu: 0 ms

=== Fungsi B (O(n^2)) ===
Total operasi: 2500
Waktu: 0 ms

=== Fungsi C (O(log n)) ===
Total iterasi: 6
Waktu: 0 ms

=== Fungsi D (O(n^3)) ===
Total operasi: 125
Waktu: 1 ms
```

Latihan 3: Class BankAccount

1. Tambahkan kode di bawah file 03-space-complexity.js.

2. Buat dua fungsi untuk menemukan dua angka dalam array yang jumlahnya = target:

▪ cariPasanganLambat(arr, target) — gunakan nested loop. Tulis Big O time & space-nya dalam komentar.

▪ cariPasanganCepat(arr, target) — gunakan Set untuk menyimpan angka yang sudah dilihat. Cek apakah target - elemen ada di Set. Tulis Big O-nya.

3. Uji kedua fungsi dengan [2, 7, 11, 15] dan target 9. Keduanya harus mengembalikan [2, 7].

4. Ukur waktu keduanya dengan array berisi 50.000 angka acak dan target yang tidak ada di array.

5. Diskusi: Fungsi mana yang lebih baik? Apa trade-off-nya?

Codenya:

```javascript
// latihan3.
function cariPasanganLambat(arr, target) {
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      if (arr[i] + arr[j] === target) {
        return [arr[i], arr[j]];
      }
    }
  }
  return null;
}

function cariPasanganCepat(arr, target) {
  let set = new Set();

  for (let num of arr) {
    let selisih = target - num;
    if (set.has(selisih)) {
      return [selisih, num];
    }
    set.add(num);
  }
  return null;
}
```

```javascript
let arrTest = [2, 7, 11, 15];
let target = 9;

console.log("Lambat:", cariPasanganLambat(arrTest, target));
console.log("Cepat:", cariPasanganCepat(arrTest, target));

let arrBesar = [];
for (let i = 0; i < 50000; i++) {
  arrBesar.push(Math.floor(Math.random() * 100000));
}

let targetTidakAda = -1;

function ukurWaktu(fn, arr, target, nama) {
  let start = Date.now();
  fn(arr, target);
  let end = Date.now();
  console.log(`${nama}: ${end - start} ms`);
}
ukurWaktu(cariPasanganCepat, arrBesar, targetTidakAda, "Cepat");
```

Hasil run:

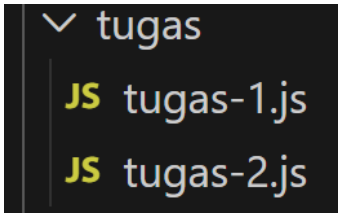
```
User@DESKTOP-VQROPFC MINGW64 /d/2025573010044_algoritma_dan_struktur_data (main)
$ cd praktikum-5

User@DESKTOP-VQROPFC MINGW64 /d/2025573010044_algoritma_dan_struktur_data/praktikum-5 (main)
$ node 03-space-complexity.js
Jumlah array : 55
Duplikasi array: [
   2,  4,  6,  8, 10,
  12, 14, 16, 18, 20
]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : 3628800
Elemen unik: 7
Lambat: [ 2, 7 ]
Cepat: [ 2, 7 ]
Cepat: 11 ms

User@DESKTOP-VQROPFC MINGW64 /d/2025573010044_algoritma_dan_struktur_data/praktikum-5 (main)
$
```

2.3 Tugas

Buat folder dan file tugas.

mkdir tugas

touch tugas/tugas-1.js tugas/tugas-2.js



Tugas 1: Analisis dan Refactor

1. Buat file tugas/tugas-1.js.

2. Terdapat tiga fungsi berikut yang perlu kamu analisis dan refactor:

3. • Fungsi A: menerima dua array, kembalikan array berisi elemen yang ada di keduanya (intersection). Implementasikan versi O(n2) dan O(n) menggunakan Set.

4. • Fungsi B: diberikan array string, kembalikan kelompok anagram. Contoh: ['eat','tea','tan','ate','nat','bat'] → [['eat','tea','ate'],['tan','nat'],['bat']]. Hint: sortir setiap kata sebagai key.

5. • Fungsi C: diberikan array angka positif, kembalikan true jika ada dua elemen yang jumlahnya sama dengan kuadrat elemen ketiga. Buat versi O(n3) dan O(n log n).

6. Untuk setiap fungsi: tulis Big O time & space dalam komentar, ukur waktu eksekusi keduanya dengan data besar.

7. Simpan dan jalankan: node tugas/tugas-1.js.

Codenya:

```javascript
function intersectionSlow(arr1, arr2) {
  const result = [];

  for (let i = 0; i < arr1.length; i++) {
    for (let j = 0; j < arr2.length; j++) {
      if (arr1[i] === arr2[j] && !result.includes(arr1[i])) {
        result.push(arr1[i]);
      }
    }
  }

  return result;
}
function intersectionFast(arr1, arr2) {
  const set = new Set(arr1);
  const result = [];

  for (let num of arr2) {
    if (set.has(num)) {
      result.push(num);
      set.delete(num);
    }
  }

  return result;
}

function groupAnagrams(arr) {
  const map = new Map();

  for (let word of arr) {
    const key = word.split("").sort().join("");

    if (!map.has(key)) {
      map.set(key, []);
    }

    map.get(key).push(word);
  }

  return Array.from(map.values());
}
function tripletSlow(arr) {
  const n = arr.length;

  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      for (let k = 0; k < n; k++) {
        if (i !== j && j !== k && i !== k) {
          if (arr[i] + arr[j] === arr[k] * arr[k]) {
            return true;
          }
        }
      }
    }
  }

  return false;
}
function tripletFast(arr) {
  arr.sort((a, b) => a - b);

  const n = arr.length;

  for (let k = n - 1; k >= 0; k--) {
    let left = 0;
    let right = k - 1;

    while (left < right) {
      const sum = arr[left] + arr[right];
      const square = arr[k] * arr[k];

      if (sum === square) {
        return true;
      } else if (sum < square) {
        left++;
      } else {
        right--;
      }
    }
  }

  return false;
}

function generateArray(size) {
  return Array.from({ length: size }, () => Math.floor(Math.random() * 1000));
}

const arr1 = generateArray(5000);
const arr2 = generateArray(5000);

console.log("=== INTERSECTION ===");
console.time("Slow");
intersectionSlow(arr1, arr2);
console.timeEnd("Slow");

console.time("Fast");
intersectionFast(arr1, arr2);
console.timeEnd("Fast");

console.log("\n=== ANAGRAM ===");
const words = ["eat", "tea", "tan", "ate", "nat", "bat"];
console.log(groupAnagrams(words));

console.log("\n=== TRIPLET ===");
const arr3 = generateArray(2000);

console.time("Triplet Slow");
tripletSlow(arr3);
console.timeEnd("Triplet Slow");

console.time("Triplet Fast");
tripletFast(arr3);
console.timeEnd("Triplet Fast");
```

Hasil run:

```
User@DESKTOP-VQROPFC MINGW64 /d/2025573010044_algoritma_dan_struktur_data/praktikum-5 (main)
$ node tugas/tugas-1.js
=== INTERSECTION ===
Slow: 53.41ms
Fast: 1.529ms

=== ANAGRAM ===
[ [ 'eat', 'tea', 'ate' ], [ 'tan', 'nat' ], [ 'bat' ] ]

=== TRIPLET ===
Triplet Slow: 0.757ms
Triplet Fast: 6.782ms
```

Tugas 2: Visualisasi Pertembuhan Big O

1. Buat file tugas/tugas-2.js.

2. Buat function benchmarkSemua(ukuranData) yang menerima array ukuran data, lalu untuk setiap ukuran, mengukur dan mencetak waktu eksekusi dari empat fungsi:

3. • fn_O1(n) → hanya return n+1

4. • fn_On(n) → loop n kali, hitung jumlah

5. • fn_OnLogn(n) → simulasikan dengan loop luar n kali, loop dalam log2(n) kali

6. • fn_On2(n) → nested loop n×n

7. Panggil benchmarkSemua([100, 500, 1000, 5000, 10000]).

8. Amati dan tuliskan komentar di kode tentang pola pertumbuhan yang kamu observasi — apakah konsisten dengan teori Big O?

9. Simpan dan jalankan: node tugas/tugas-2.js.

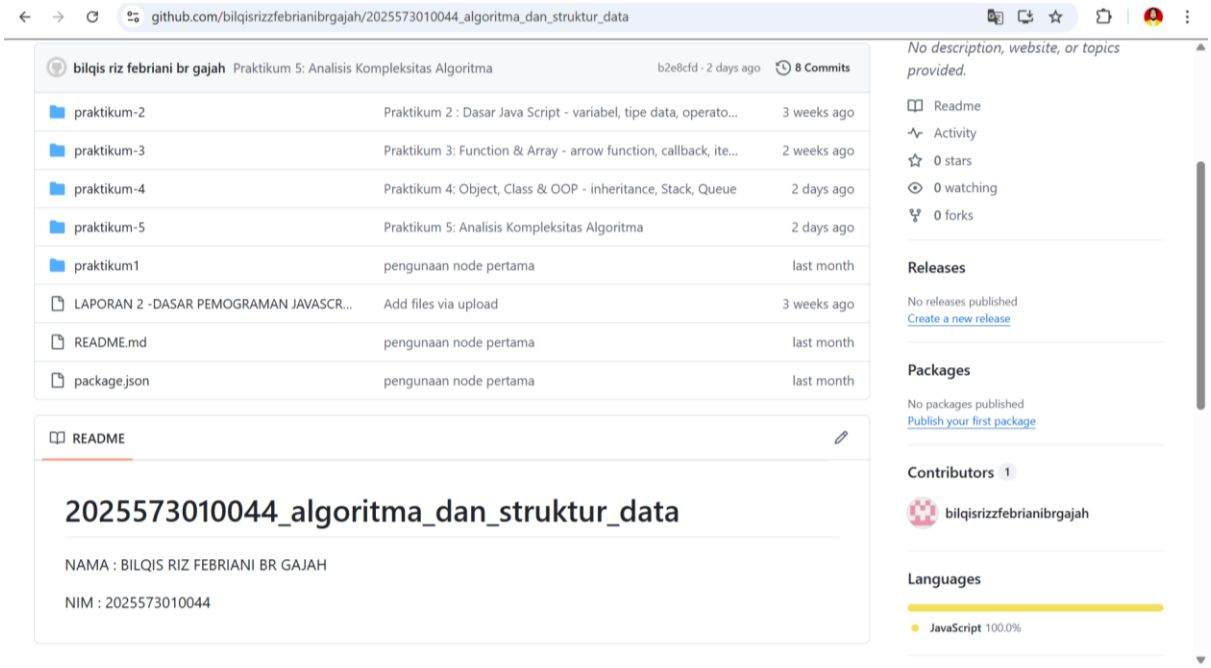
Codenya:

```javascript
function fn_O1(n) {
  return n + 1;
}
function fn_On(n) {
  let sum = 0;
  for (let i = 0; i < n; i++) {
    sum += i;
  }
  return sum;
}
function fn_OnLogn(n) {
  let count = 0;
  const log = Math.floor(Math.log2(n));

  for (let i = 0; i < n; i++) {
    for (let j = 0; j < log; j++) {
      count++;
    }
  }

  return count;
}
function fn_On2(n) {
  let count = 0;

  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      count++;
    }
  }

  return count;
}

function benchmarkSemua(ukuranData) {
  for (let n of ukuranData) {
    console.log(`\n=== n = ${n} ===`);

    console.time("O(1)");
    fn_O1(n);
    console.timeEnd("O(1)");

    console.time("O(n)");
    fn_On(n);
    console.timeEnd("O(n)");

    console.time("O(n log n)");
    fn_OnLogn(n);
    console.timeEnd("O(n log n)");

    console.time("O(n^2)");
    fn_On2(n);
    console.timeEnd("O(n^2)");
  }
}
benchmarkSemua([100, 500, 1000, 5000, 10000]);
```

Hasil run:

```
User@DESKTOP-VQROPFC MINGW64 /d/2025573010044_algoritma_dan_struktur_data (main)
$ cd praktikum-5

User@DESKTOP-VQROPFC MINGW64 /d/2025573010044_algoritma_dan_struktur_data/praktikum-5 (main)
$ node tugas/tugas-2.js

=== n = 100 ===
O(1): 0.026ms
O(n): 0.044ms
O(n log n): 0.413ms
O(n^2): 1.534ms

=== n = 500 ===
O(1): 0.018ms
O(n): 0.019ms
O(n log n): 0.094ms
O(n^2): 13.036ms

=== n = 1000 ===
O(1): 0.016ms
O(n): 0.047ms
O(n log n): 0.092ms
O(n^2): 2.752ms

=== n = 5000 ===
O(1): 0.008ms
O(n): 0.011ms
O(n log n): 0.158ms
O(n^2): 42.429ms

=== n = 10000 ===
O(1): 0.005ms
O(n): 0.025ms
O(n log n): 0.226ms
O(n^2): 113.604ms
```

semua latihan dan tugas di commit dan push ke repository GitHub



BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa pemahaman terhadap kompleksitas algoritma sangat penting dalam pengembangan perangkat lunak. Dengan mengetahui kompleksitas waktu dan ruang, seorang programmer dapat memilih algoritma yang paling efisien sesuai dengan kebutuhan.

Dari percobaan yang dilakukan, terlihat bahwa algoritma dengan kompleksitas lebih rendah seperti O(n) atau O(log n) memiliki performa yang jauh lebih baik dibandingkan algoritma dengan kompleksitas tinggi seperti $O(n^2)$ atau $O(n^3)$, terutama saat data yang digunakan semakin besar.

Selain itu, penggunaan struktur data seperti Set juga dapat membantu meningkatkan efisiensi algoritma. Oleh karena itu, penting bagi seorang programmer untuk tidak hanya membuat program yang berjalan, tetapi juga mempertimbangkan efisiensi dan optimalisasi kode.